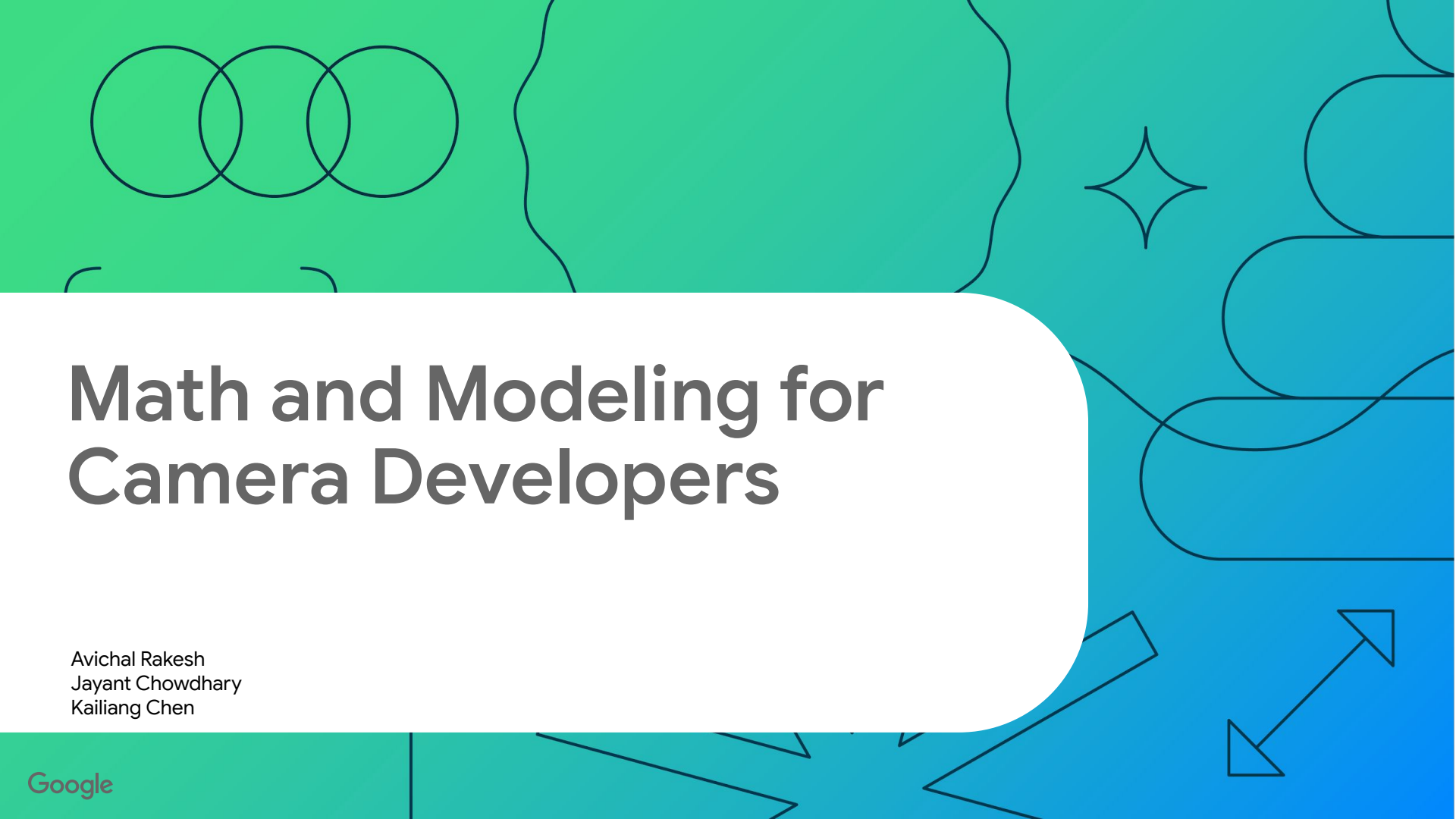




Math and Modeling for Camera Developers

Avichal Rakesh
Jayant Chowdhary
Kailiang Chen

How do these work!?



[Source](#)



[Source](#)

Prerequisites for the talk

- Basics of Linear Algebra / Matrix Math – nothing too complex!
 - There are more resources at the end if you'd like a refresher (or a deep dive).
- Some Camera Experience (Android or otherwise)
 - Not required, but will be helpful context.
- For extra credits: Some Graphics experience.

Agenda

1. Camera Intrinsics
 - Making an image from the world.
2. Camera Extrinsics & Applications
 - Making the world from image(s).
3. Android APIs
 - Tools in the shed.

Camera Intrinsics

(Making an image from the world)

‘Capturing Image’ as a Mathematical Operation



+



=



The formal name of the mathematical operation is **Projection**.

pro·jec·tion

[/prəˈjekSH(ə)n/] **noun**

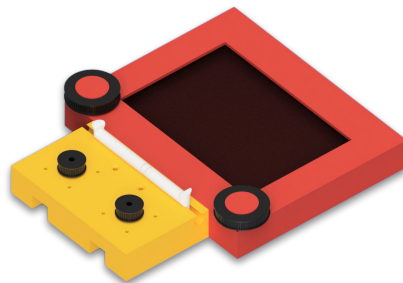
In mathematics, a projection is an idempotent mapping of a set (or other mathematical structure) into a subset (or sub-structure).

'Capturing Image' as a Mathematical Operation

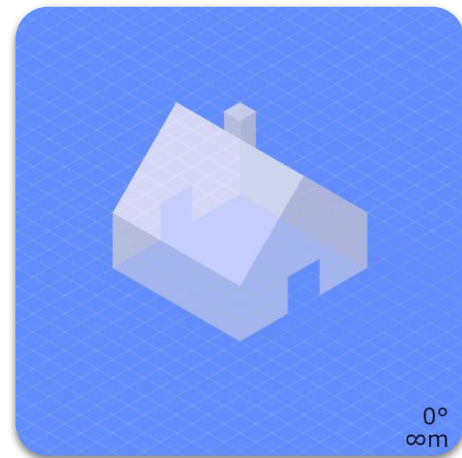
*Perspective
Projection



Orthographic
Projection

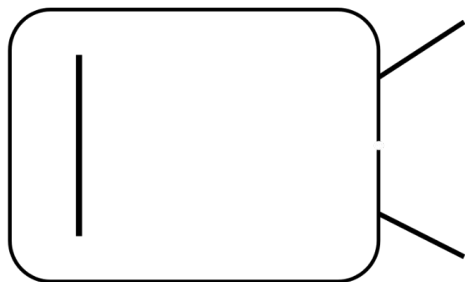


...not as different
as they might
seem.



Pinhole Camera Model in 2D

(an Intuitive Image)



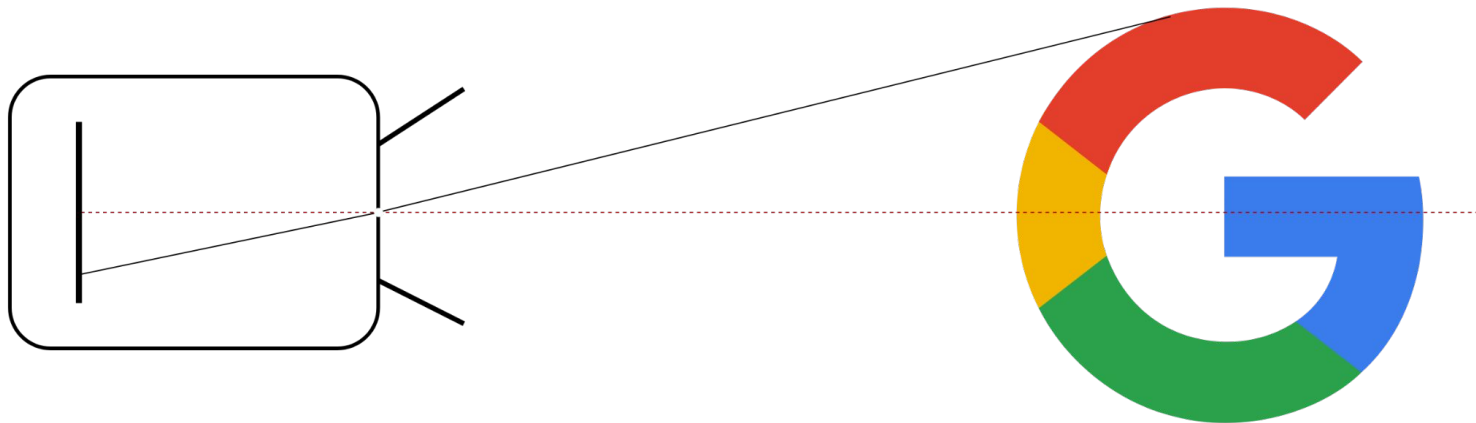
Pinhole Camera Model in 2D

(an Intuitive Image)



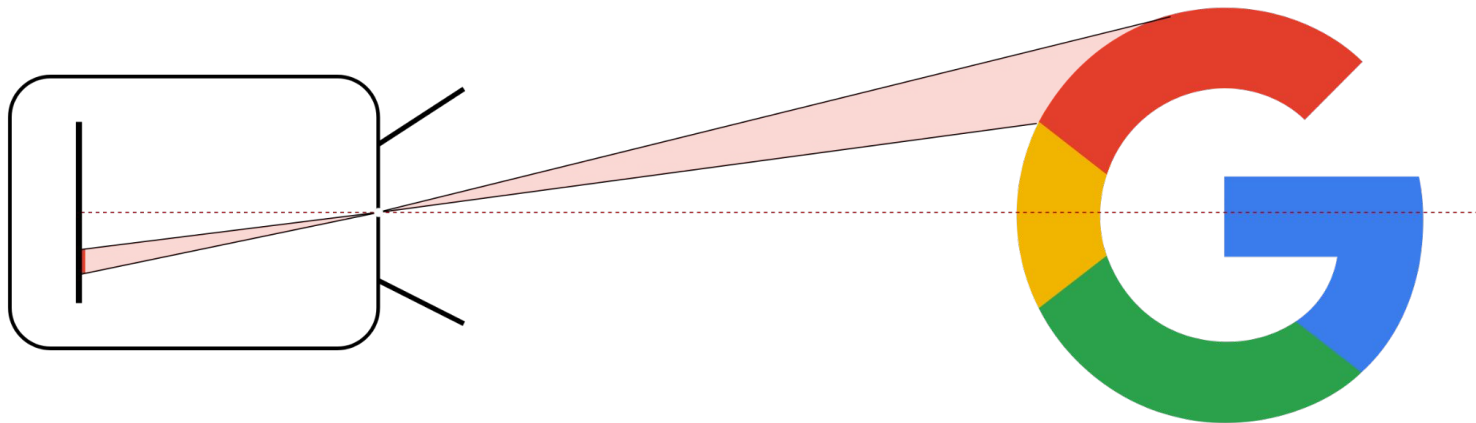
Pinhole Camera Model in 2D

(an Intuitive Image)



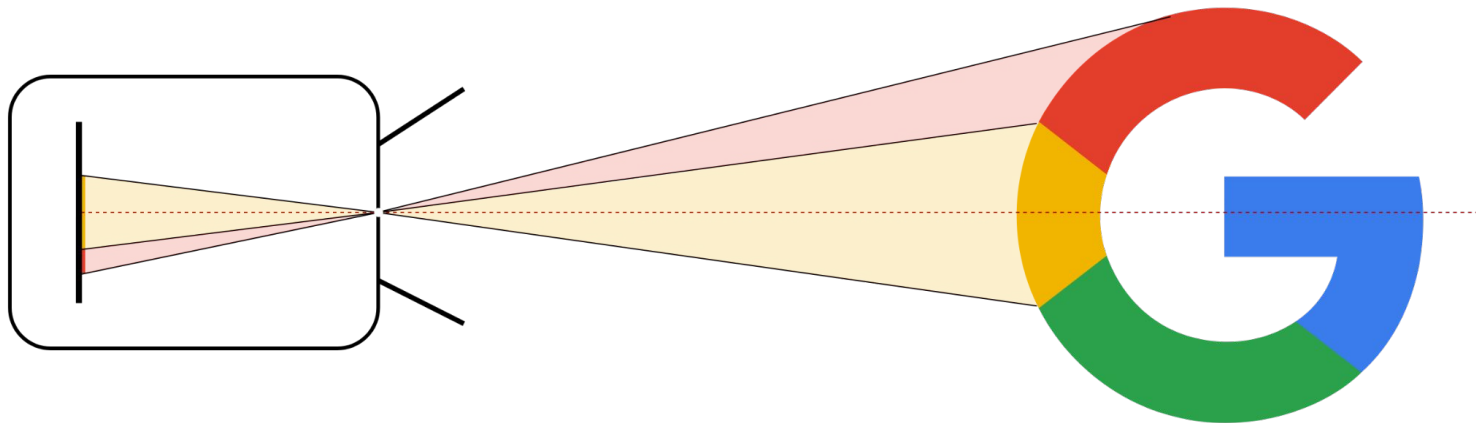
Pinhole Camera Model in 2D

(an Intuitive Image)



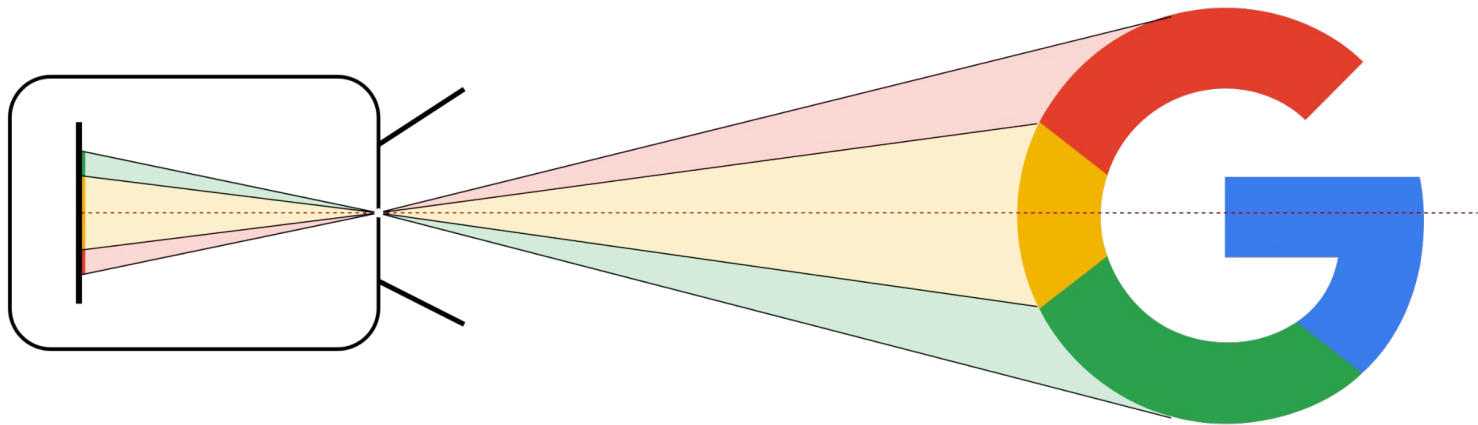
Pinhole Camera Model in 2D

(an Intuitive Image)



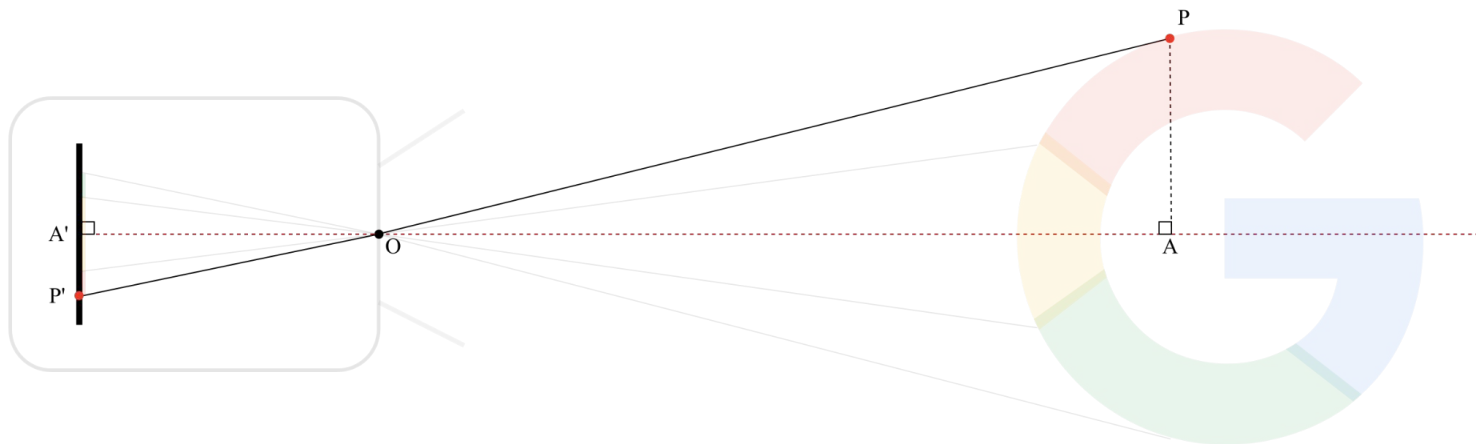
Pinhole Camera Model in 2D

(an Intuitive Image)



Pinhole Camera Model in 2D

(the Math)



Pinhole Camera Model in 2D

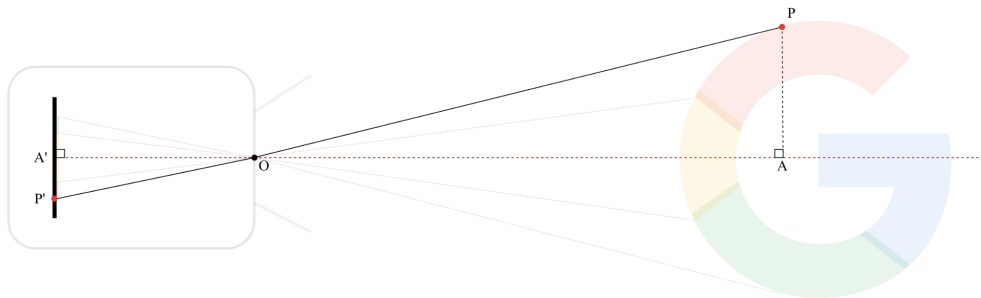
(the Math)

$$\angle P'A'O = \angle PAO = 90^\circ$$

$$\angle P'OA' = \angle POA$$

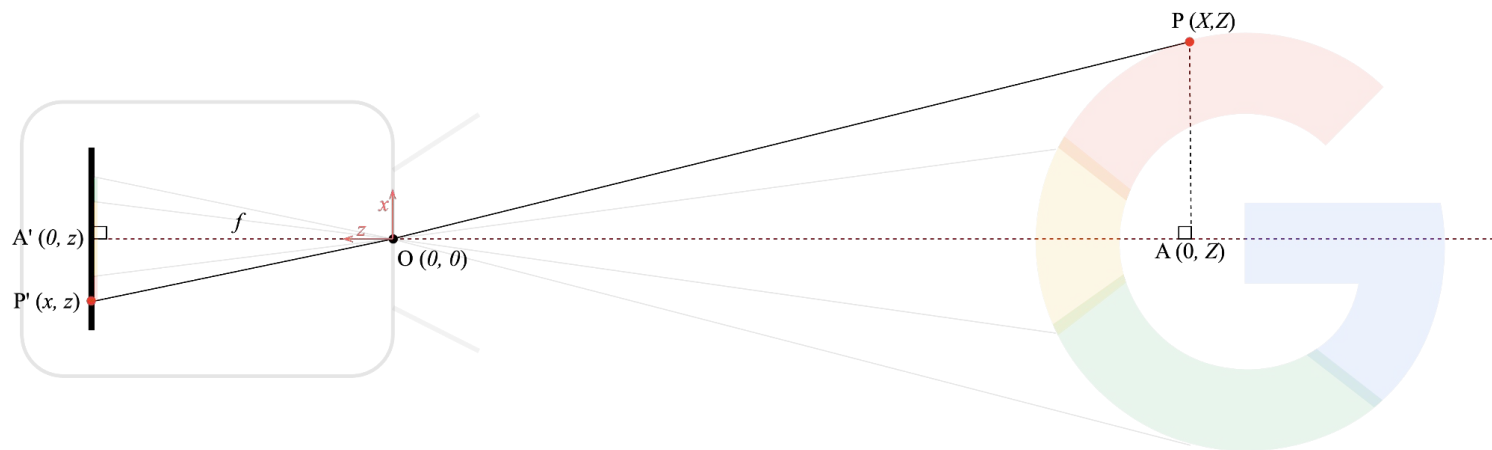
$$\therefore \angle A'P'O = \angle APO$$

$$\Rightarrow \triangle OA'P' \simeq \triangle OAP$$



Pinhole Camera Model in 2D

(the Math)



Pinhole Camera Model in 2D

(the Math)

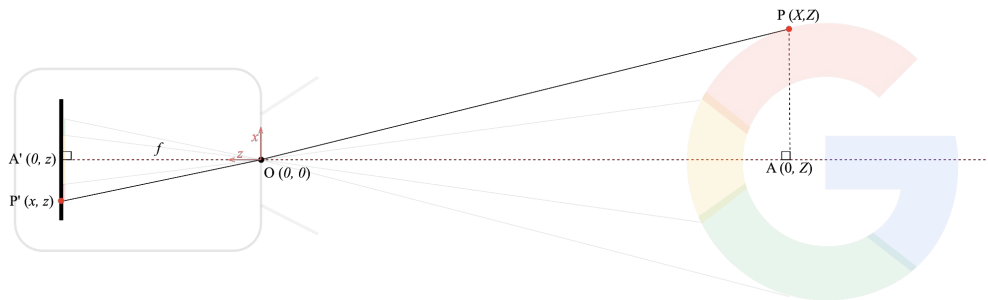
$$\frac{\overline{A'P'}}{\overline{AP}} = \frac{\overline{OA'}}{\overline{OA}}$$

$$\Rightarrow \frac{\overline{A'P'}}{X} = \frac{f}{Z}$$

$$\Rightarrow \overline{A'P'} = \frac{f}{Z} \cdot X$$

$$\therefore x = \frac{X}{Z} \cdot f$$

$$P' = \left(\frac{X}{Z} \cdot f, f \right)$$



Pinhole Camera Model

(extending to 3D)

Surprisingly, the math remains exactly the same in 3D.
Simply calculate xz and yz planes separately!

So, for a point $P(X, Y, Z)$, the image will be formed at $P'(x, y)$
where:

$$x = \frac{X}{Z} \cdot f \qquad y = \frac{Y}{Z} \cdot f$$

Nice and simple. Let's complicate it to make it even easier!

Homogeneous Coordinates

Special representation to make coordinate values invariant to elementwise scaling.

$$P := \begin{bmatrix} x \\ y \end{bmatrix} \xrightarrow{\text{becomes}} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \sim \begin{bmatrix} kx \\ ky \\ k \end{bmatrix}$$

Consider it the imaginary numbers of linear algebra. It exists to make certain math easier.

Homogeneous Coordinates

How does it help us?

The goal is to find a constant matrix $\mathbf{C}$, such that

$$\vec{P'} = \begin{bmatrix} \frac{X}{Z} \cdot f \\ \frac{Y}{Z} \cdot f \end{bmatrix} = \mathbf{C} \cdot \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

Naively, this operation cannot be done.

Homogeneous Coordinates

How does it help us?

Homogeneous Coordinates to the rescue.

$$\begin{aligned} \vec{P'} &= \begin{bmatrix} \frac{X}{Z} \cdot f \\ \frac{Y}{Z} \cdot f \\ 1 \end{bmatrix} \xrightarrow{\text{becomes}} \begin{bmatrix} \frac{X}{Z} \cdot f \\ \frac{Y}{Z} \cdot f \\ 1 \end{bmatrix} \\ &\xrightarrow{\text{becomes}} \begin{bmatrix} X \cdot f \\ Y \cdot f \\ Z \end{bmatrix} \end{aligned}$$

Homogeneous Coordinates

How does it help us?

Homogeneous Coordinates to the rescue.

$$\begin{bmatrix} X \cdot f \\ Y \cdot f \\ Z \end{bmatrix} = \mathbf{C} \cdot \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

It's a simple scaling operation. Trivial to solve!

Homogeneous Coordinates

How does it help us?

Homogeneous Coordinates to the rescue.

$$\vec{P'} = \begin{bmatrix} x \\ y \\ t \end{bmatrix} = \begin{bmatrix} X \cdot f \\ Y \cdot f \\ Z \end{bmatrix} = \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

$\overset{C}{\text{(constant)}}$

The complicated math of projection is now a simple scaling!

What about “real” cameras?

21st century cameras have lenses, not pinholes, and these lenses can often move. How do we accommodate that?

Lenses add a few more complications over the pinhole model:

1. Principal/Optical Axis Translation
2. Sensor Skew
3. Lens Distortion/Aberration – no exact mathematical model.
 - Can be approximated by some well-known models.
 - Generally handled by the ISP/Camera Manufacturer.

What about “real” cameras?

Optical Axis Translation:

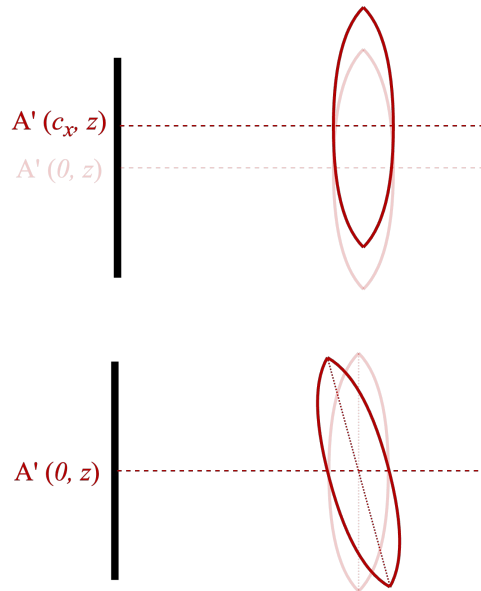
Decouples the sensor/image coordinates from the pinhole coordinates.

Simply translate P' once it is on the sensor

Sensor Skew:

Handles cases where the lens plane and the sensor plane are not parallel.

Just pretend the whole world was sheared before capturing the image!



Thankfully, they’re both simple operations!

What about “real” cameras?

Optical Axis Translation and Sensor Skew

$$\vec{P'} = \underbrace{\begin{bmatrix} 1 & 0 & c_x \\ 0 & 1 & c_y \\ 0 & 0 & 1 \end{bmatrix}}_{\text{translate}} \cdot \underbrace{\begin{bmatrix} f_x & 0 & 0 \\ 0 & f_y & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{\text{scale}} \cdot \underbrace{\begin{bmatrix} 1 & \frac{s}{f_x} & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{\text{skew}} \cdot \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

In homogeneous coordinates. 

$$\vec{P'} = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

**Camera
Intrinsics!**

What about “real” cameras?

Optical Axis Translation and Sensor Skew

$$\vec{P'} = \underbrace{\begin{bmatrix} 1 & 0 & c_x \\ 0 & 1 & c_y \\ 0 & 0 & 1 \end{bmatrix}}_{\text{translate}} \cdot \underbrace{\begin{bmatrix} f_x & 0 & 0 \\ 0 & f_y & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{\text{scale}} \cdot \underbrace{\begin{bmatrix} 1 & \frac{s}{f_x} & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{\text{skew}} \cdot \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

$$\vec{P'} = \begin{bmatrix} x \\ y \\ t \end{bmatrix} = \begin{bmatrix} f_x & s & c_x & 0 \\ 0 & f_y & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

**Camera
Intrinsics!**
(in homogeneous coordinates)

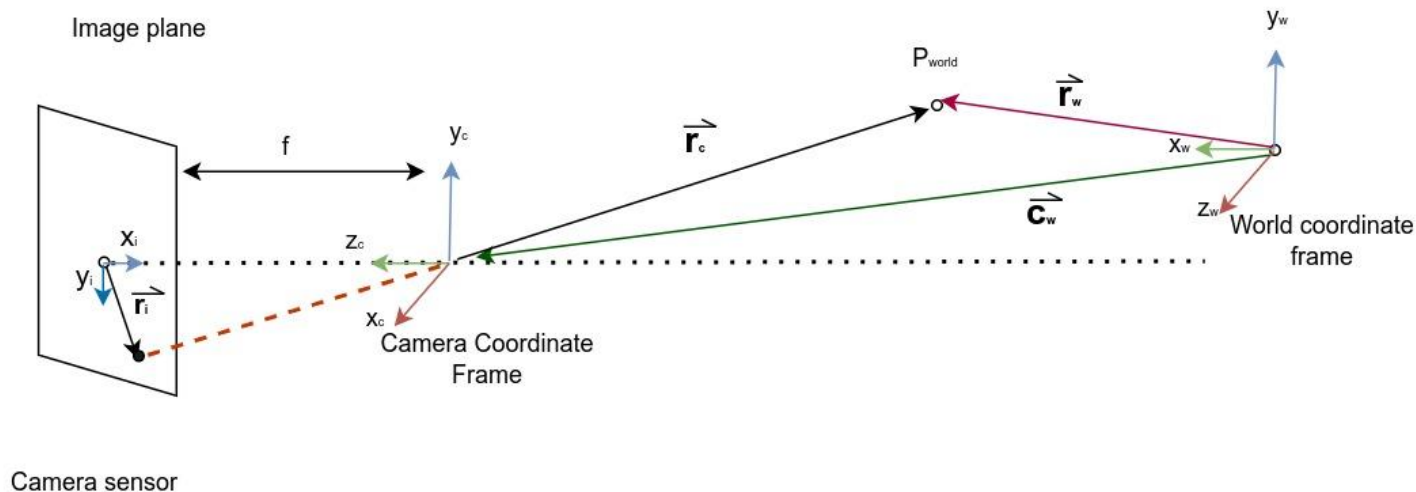
Camera Extrinsics (and Applications)

Camera Extrinsic parameters

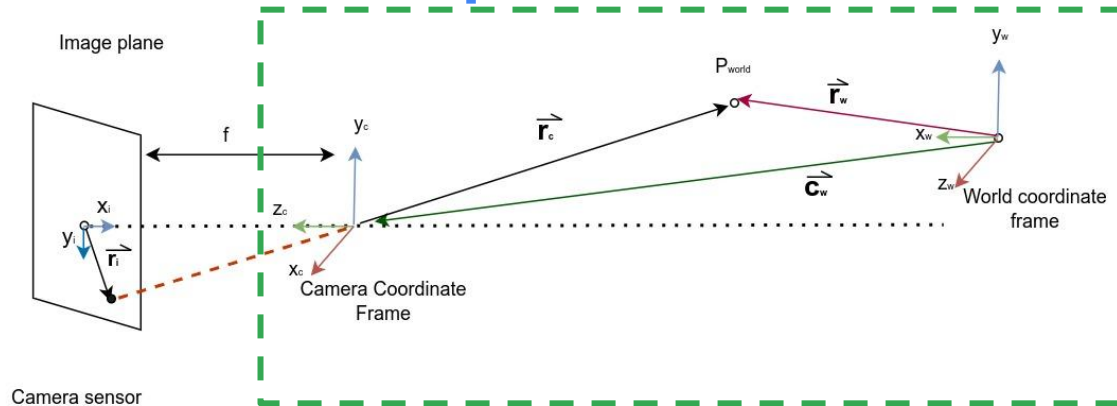
- Ultimately, we want to be able to map a point 3D $\rightarrow$ 2D (sensor space) from a reference point in which is NOT the camera's coordinate frame
- This also helps us work backwards from a point in the sensor coordinate system to the world coordinate system as we will see later (given depth)

Camera Extrinsic parameters

- We have a way to convert 3D coordinate in camera coordinate frame to image / sensor coordinates
- However, the true transform we need is : $\vec{r}_w \Rightarrow \vec{r}_i$



Camera Extrinsic parameters



$$\begin{bmatrix} x_i \\ y_i \end{bmatrix}$$



$$\begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix}$$



$$\begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix}$$

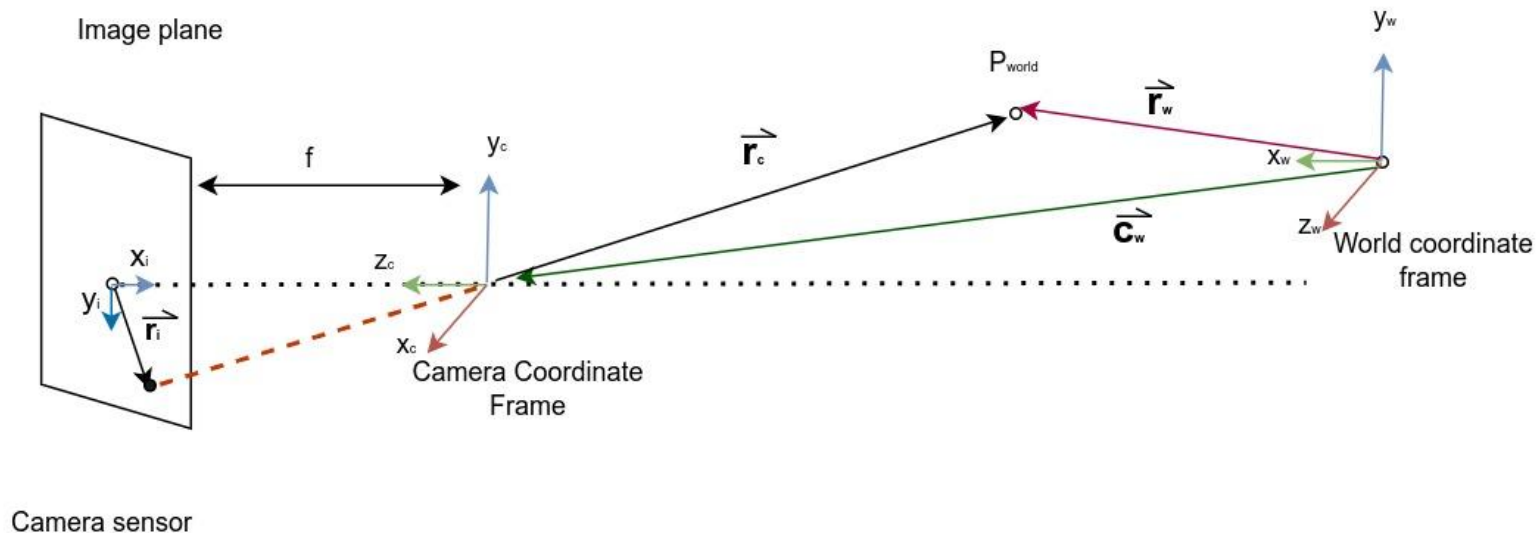
Image / sensor coordinate system

Camera coordinate system

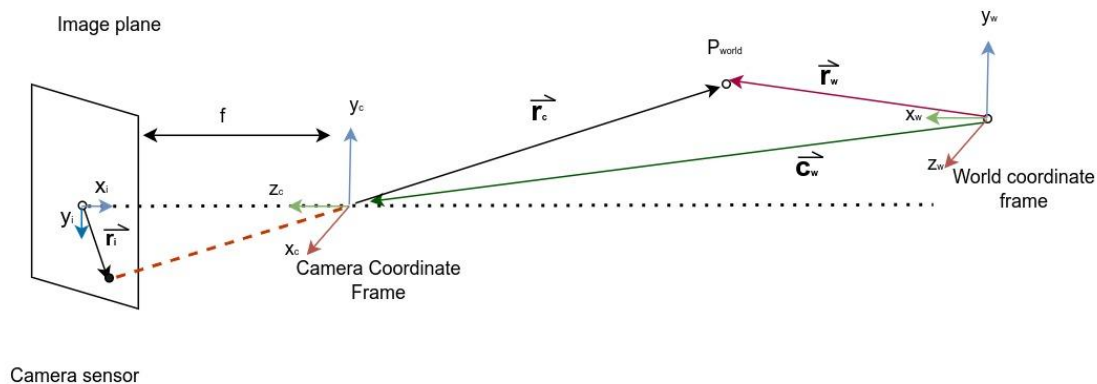
World coordinate system

Camera Extrinsic parameters

- $\mathbf{r}_w \Rightarrow \mathbf{r}_i$ naturally, doesn't only depend on the camera's internal geometry
- Also depends on the camera's position in space w.r.t world coordinate frame



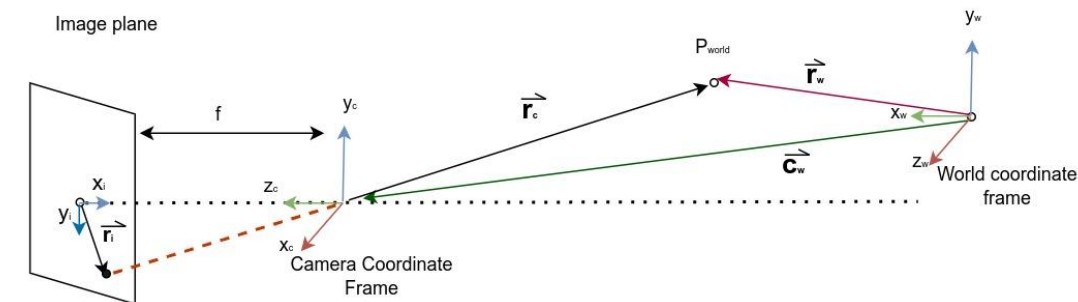
Camera Extrinsic parameters



In any coordinate system, vector equation for transformation:

$$\vec{r}_c = \vec{r}_w - \vec{c}_w$$

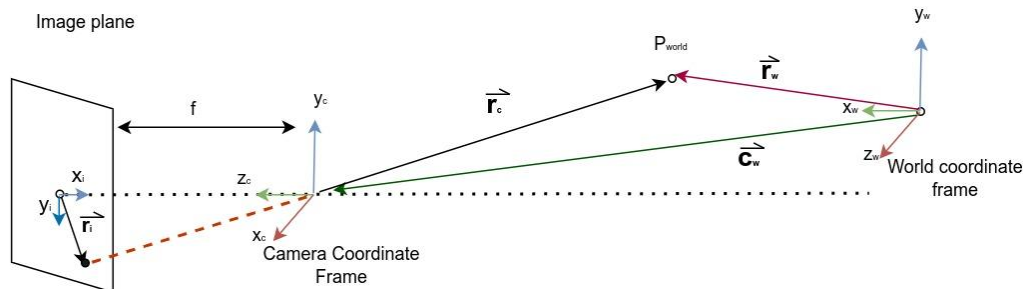
Camera Extrinsic parameters



Camera sensor

- We want $\vec{r}_c$ in terms of the camera coordinate system
- We know $\vec{r}_w$ and $\vec{c}_w$ in terms of the world coordinate system - however we need to write the vector equation in the same coordinate system.

Camera Extrinsic parameters



Camera sensor

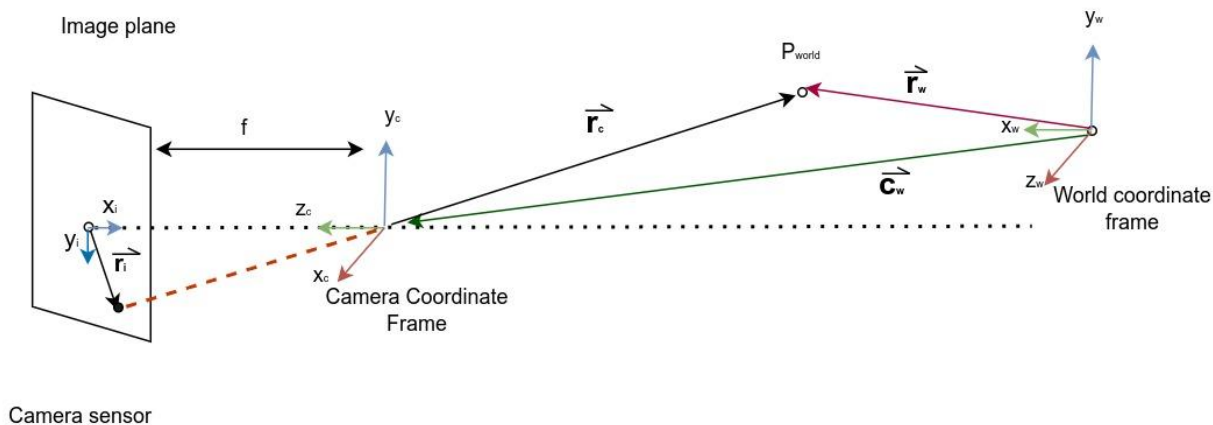
- To express $\vec{r}_c$ in terms of camera coordinate system, we use a transformation matrix to account for rotation and translation

$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

$r_{11}, r_{12}, r_{13} \equiv x_c$ expressed in terms of x_w, y_w and z_w and similar for other r_{ij}

The rows are orthonormal - dot product is zero

Camera Extrinsic parameters



$$\vec{r}_c = \mathbf{R}(\vec{r}_w - \vec{c}_w)$$

Camera coordinate system

World coordinate system

Camera Extrinsic parameters

$$\vec{\mathbf{r}}_{\mathbf{c}} = \mathbf{R}(\vec{\mathbf{r}}_{\mathbf{w}} - \mathbf{c}_{\mathbf{w}})$$



$$\vec{\mathbf{r}}_{\mathbf{c}} = \mathbf{R}\vec{\mathbf{r}}_{\mathbf{w}} - \mathbf{R}\mathbf{c}_{\mathbf{w}}$$



$$\vec{\mathbf{r}}_{\mathbf{c}} = \mathbf{R}\vec{\mathbf{r}}_{\mathbf{w}} + \mathbf{t}$$

Camera Extrinsic parameters

- Use homogeneous coordinates to use a single transform

$$r_c = \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} tx \\ ty \\ tz \\ 1 \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

Combined Camera Projection Matrix

$$\begin{bmatrix} x_i \\ y_i \\ t \end{bmatrix} = \begin{bmatrix} f_x & s & c_x & 0 \\ 0 & f_y & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

Sensor coordinates

Intrinsic parameters

Extrinsic parameters

World frame coordinates



$$\begin{bmatrix} x_i \\ y_i \\ t \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

Projection matrix

Camera Calibration : Estimating Projection Matrix

Goal : Estimate the projection matrix of a given camera

Method: Get images of an object of known geometry and plug feature coordinates into projection matrix. Then solve for unknown parameters.

Camera Calibration : Estimating Projection Matrix

- We capture an image of an object of known geometry.
- We know sensor coordinates of certain landmarks
- We know world coordinates of those landmarks.
- Solve / estimate projection matrix!



Note: Image generated by Gemini

Camera Calibration : Estimating Projection Matrix

Image coordinates

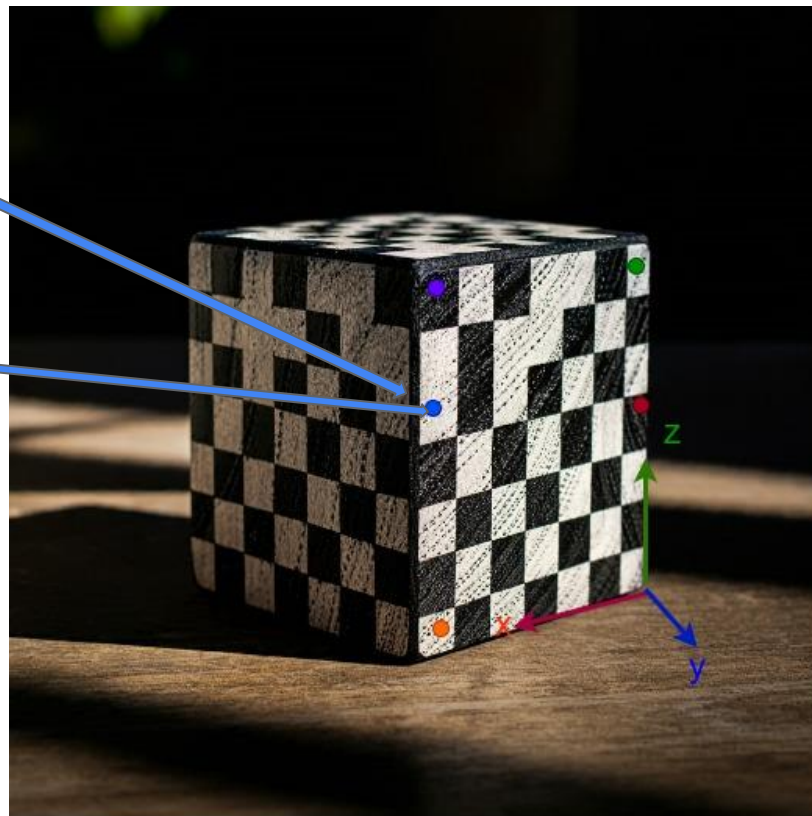
Projection matrix

World coordinates

$$\begin{bmatrix} x_i^j \\ y_i^j \\ 1 \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \begin{bmatrix} x_w^j \\ y_w^j \\ z_w^j \\ 1 \end{bmatrix}$$

known unknown known

- 12 unknowns and each point gives us 2 equations - so 6 non coplanar points are enough for calibration



Intrinsics and Extrinsics : Special Properties

$$\begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Intrinsic parameters

$$\begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} t_x \\ t_y \\ t_z \\ 1 \end{bmatrix}$$

extrinsic parameters

- Extrinsic matrix: Rotation matrix rows are orthonormal
- Intrinsic matrix: Upper right triangle matrix - elements below diagonal are 0

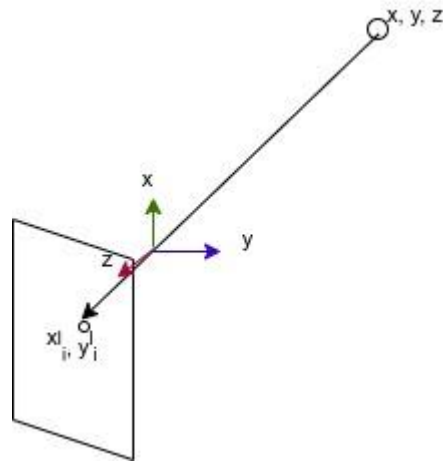
Camera Calibration : Estimating Projection Matrix

$$\begin{bmatrix} x_i^j \\ y_i^j \\ 1 \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \begin{bmatrix} x_w^j \\ y_w^j \\ z_w^j \\ 1 \end{bmatrix}$$

- We can break up the projection matrix into intrinsic matrix and extrinsic matrix due to special properties Split into extrinsic matrix and intrinsic matrix using [QR decomposition](#)

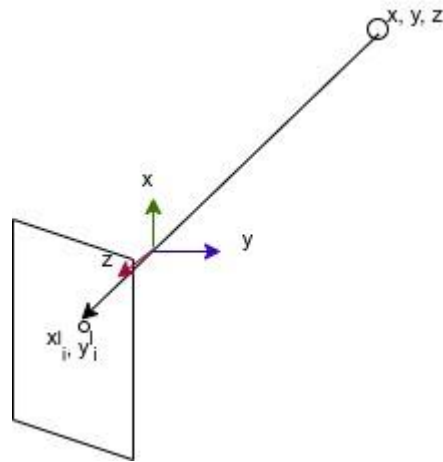
Depth estimation using stereo camera

- Scenario : you have a camera with known intrinsic parameters
- **Question** : Can we know the corresponding 3D coordinates for a point in sensor coordinates (2D) ?



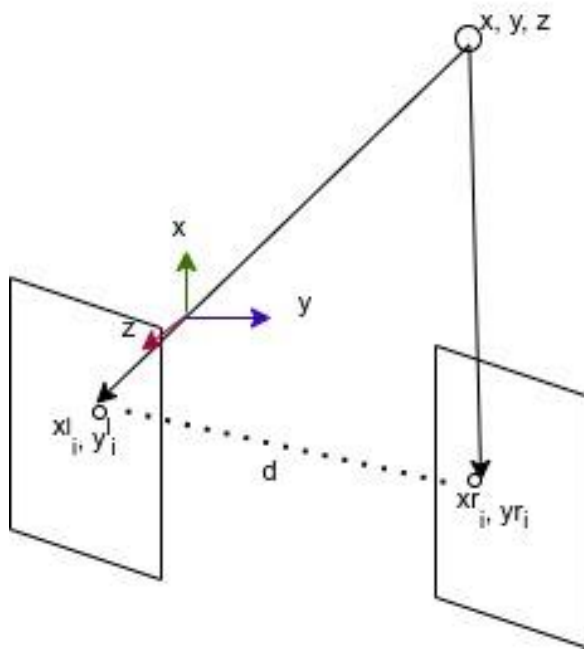
Depth estimation using stereo

- Scenario : you have a camera with known intrinsic parameters
- **Question** : Can we know the corresponding 3D coordinates for a 2D point in sensor coordinates ?
- **Answer** : We cannot estimate depth using just one image since a point in the image can correspond to **any** point on a ray connecting the pinhole and point and image



Depth estimation using stereo

- We cannot estimate depth using just one image since a point in the image can correspond to **any** point on a ray connecting the pinhole and point and image
- Solution : Capture images from different points (horizontally displaced) of view and use triangulation



d : horizontal displacement between the two camera coordinate systems

$$z = d * (f_x) / (x_i^r - x_i^l)$$

3D Reconstruction

- **3D reconstruction:** create a 3D representation of a scene using multi-view stereo, structure from motion etc.,.
- **Structure from motion:** reconstruct the camera poses and 3D structure of a scene from a set of 2D images.



Structure from Motion

Camera
Calibration

Feature
Extraction

Feature
Matching

Pose
Estimation

Triangulation

Bundle
Adjustment



Structure from Motion

Camera
Calibration

Feature
Extraction

Feature
Matching

Pose
Estimation

Triangulation

Bundle
Adjustment



Structure from Motion

Camera
Calibration

Feature
Extraction

Feature
Matching

Pose
Estimation

Triangulation

Bundle
Adjustment



Structure from Motion

Camera
Calibration

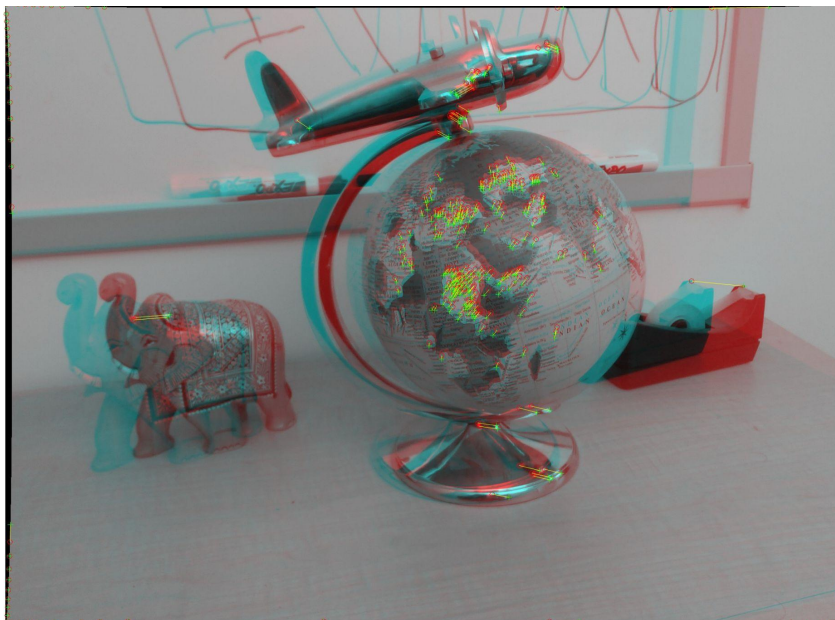
Feature
Extraction

Feature
Matching

Pose
Estimation

Triangulation

Bundle
Adjustment



Structure from Motion

Camera
Calibration

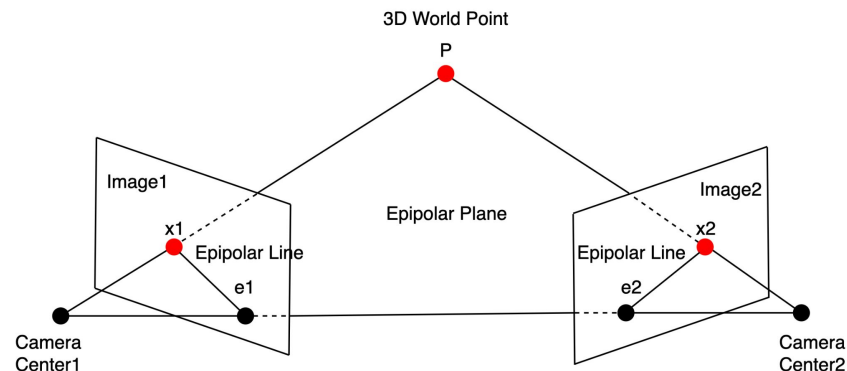
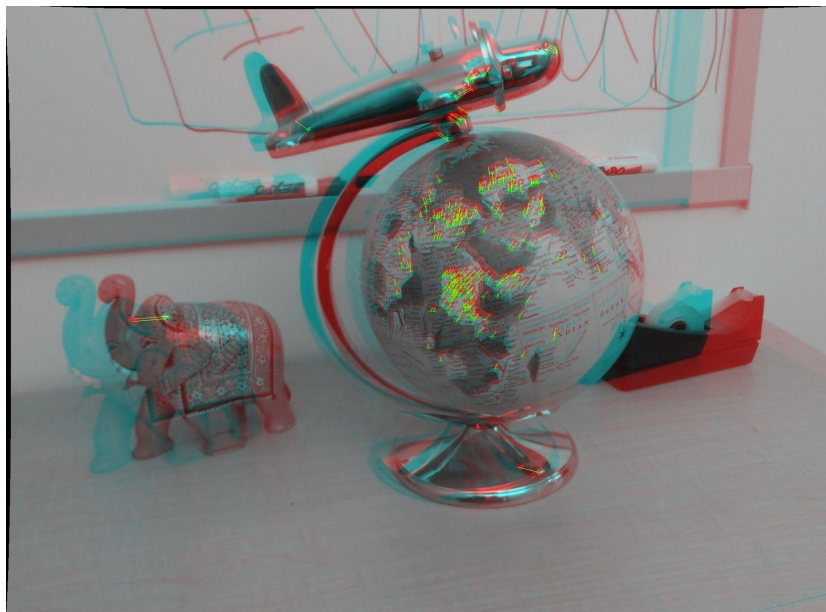
Feature
Extraction

Feature
Matching

Pose
Estimation

Triangulation

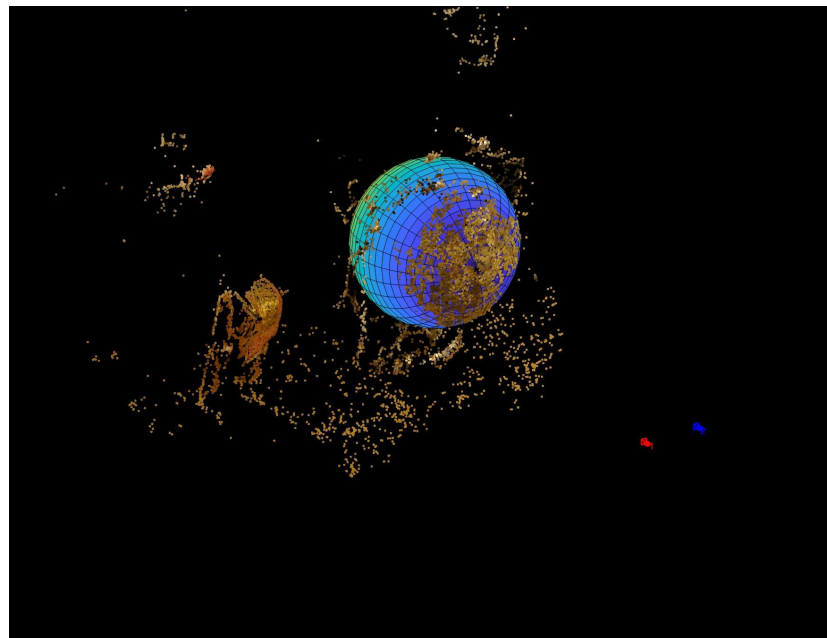
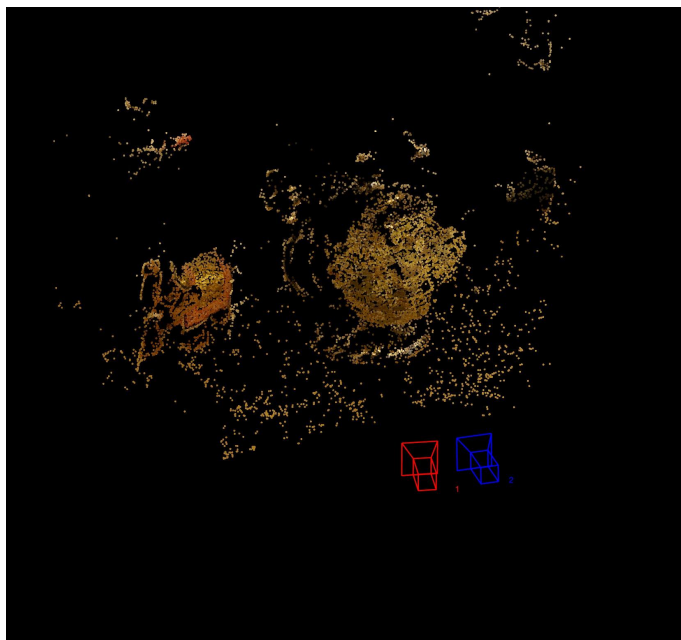
Bundle
Adjustment



[What are Essential Matrix and Epipolar Constraint](#)

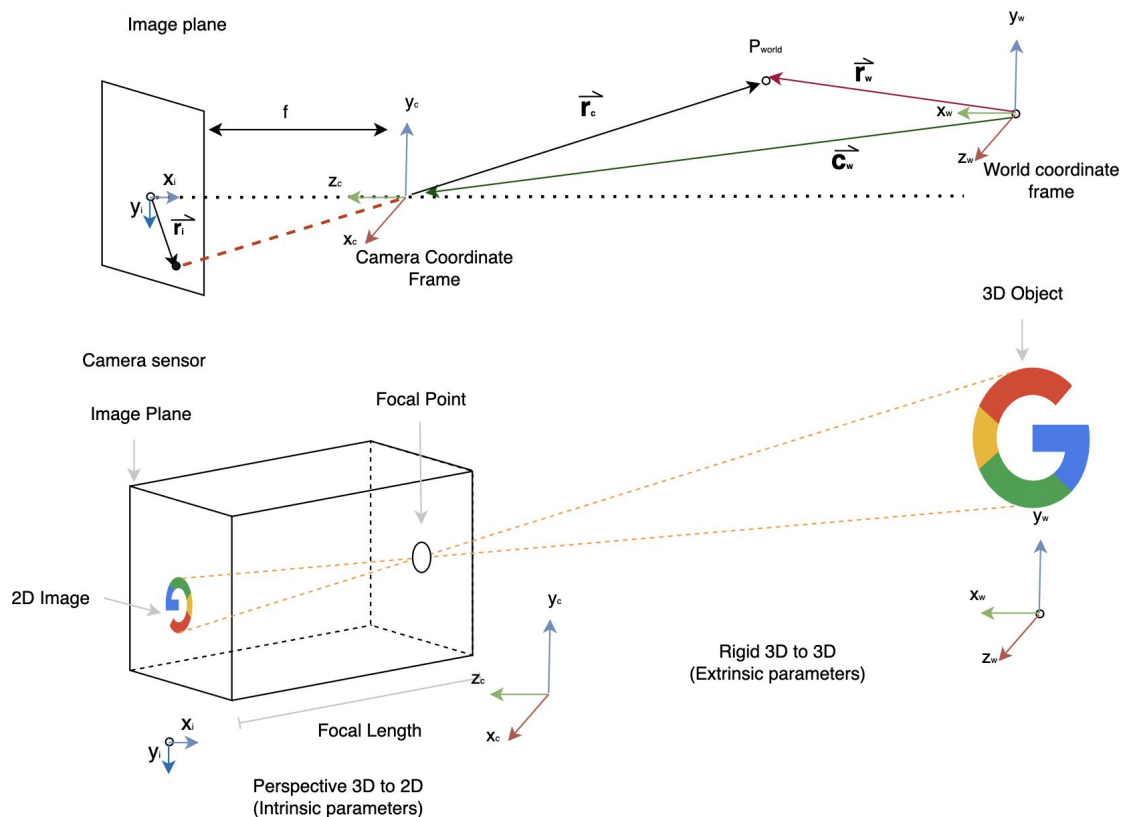
[Source](#)

Structure from Motion



Android APIs

World (3D) - Camera (3D) - Sensor (2D)



Android Camera2 APIs - Intrinsics

Camera calibration = intrinsics + extrinsics + distortion coefficients

```
public static final Key<float[]> LENS_INTRINSIC_CALIBRATION  
[ $f_x$ ,  $f_y$ ,  $c_x$ ,  $c_y$ ,  $s$ ]
```

Generalized pinhole camera model

$$\mathbf{P} = \mathbf{K} [\mathbf{R} \mathbf{t}]$$


$$\mathbf{P} = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 - 2y_r^2 - 2z_r^2 & 2x_r y_r - 2z_r w_r & 2x_r z_r + 2y_r w_r & x_t \\ 2x_r y_r + 2z_r w_r & 1 - 2x_r^2 - 2z_r^2 & 2y_r z_r - 2x_r w_r & y_t \\ 2x_r z_r - 2y_r w_r & 2y_r z_r + 2x_r w_r & 1 - 2x_r^2 - 2y_r^2 & z_t \end{bmatrix}$$

Intrinsic parameters

Extrinsic parameters

Android Camera2 APIs - Extrinsics

`public static final Key<Integer> LENS_POSE_REFERENCE` (the origin in world coordinate)

Possible Values:

- [PRIMARY_CAMERA](#)
- [GYROSCOPE](#)
- [UNDEFINED](#)
- [AUTOMOTIVE](#)

$$\mathbf{P} = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 - 2y_r^2 - 2z_r^2 & 2x_r y_r - 2z_r w_r & 2x_r z_r + 2y_r w_r & x_t \\ 2x_r y_r + 2z_r w_r & 1 - 2x_r^2 - 2z_r^2 & 2y_r z_r - 2x_r w_r & y_t \\ 2x_r z_r - 2y_r w_r & 2y_r z_r + 2x_r w_r & 1 - 2x_r^2 - 2y_r^2 & z_t \end{bmatrix}$$

Intrinsic parameters

Extrinsic parameters

`public static final Key<float[]> LENS_POSE_ROTATION`

`[ $x_r, y_r, z_r, w_r$ ]` ([quaternion to rotation matrix](#))

`public static final Key<float[]> LENS_POSE_TRANSLATION`

`[ $x_t, y_t, z_t$ ]`

Android Camera2 APIs - Distortion

Lens distortion = Radial + Tangential

Radial distortion: occurs when light rays bend more near the edges of a lens than they do at its center.

Tangential distortion: occurs when the lens and the image plane are not parallel.

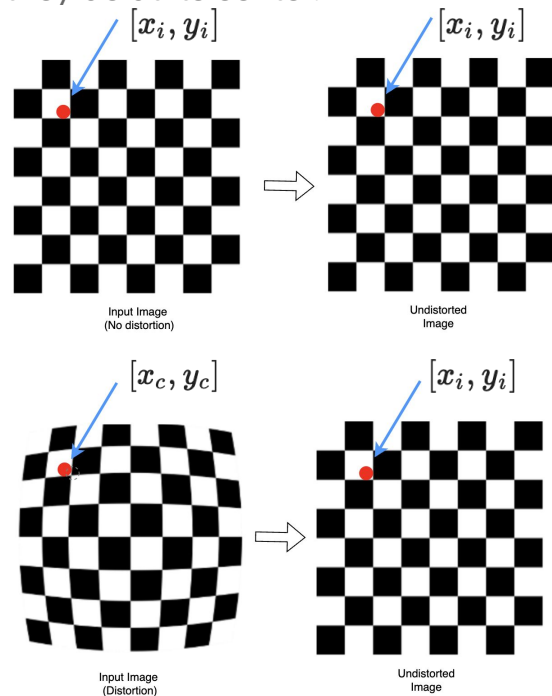
The distortion model used is the [Brown-Conrady model](#).

```
public static final Key<float[]> LENS_DISTORTION  
[ $\kappa_1, \kappa_2, \kappa_3, \kappa_4, \kappa_5$ ]
```

Radial : [$\kappa_1, \kappa_2, \kappa_3$]

Tangential : [κ_4, κ_5]

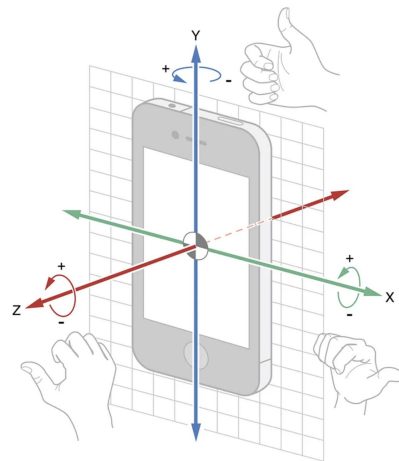
$$(x_c, y_c) = f(x_i, y_i, \kappa_1, \kappa_2, \kappa_3, \kappa_4, \kappa_5)$$



Camera + IMU

IMU (Inertial Measurement Unit), sensors that measure the device's motion and orientation in 3D space.

- [Accelerometer](#): Measures linear acceleration in $[x, y, z]$ - (movement, tilt, and gravity)
- [Gyroscope](#): Measures angular velocity around $[x, y, z]$ - (roll, pitch, yaw)



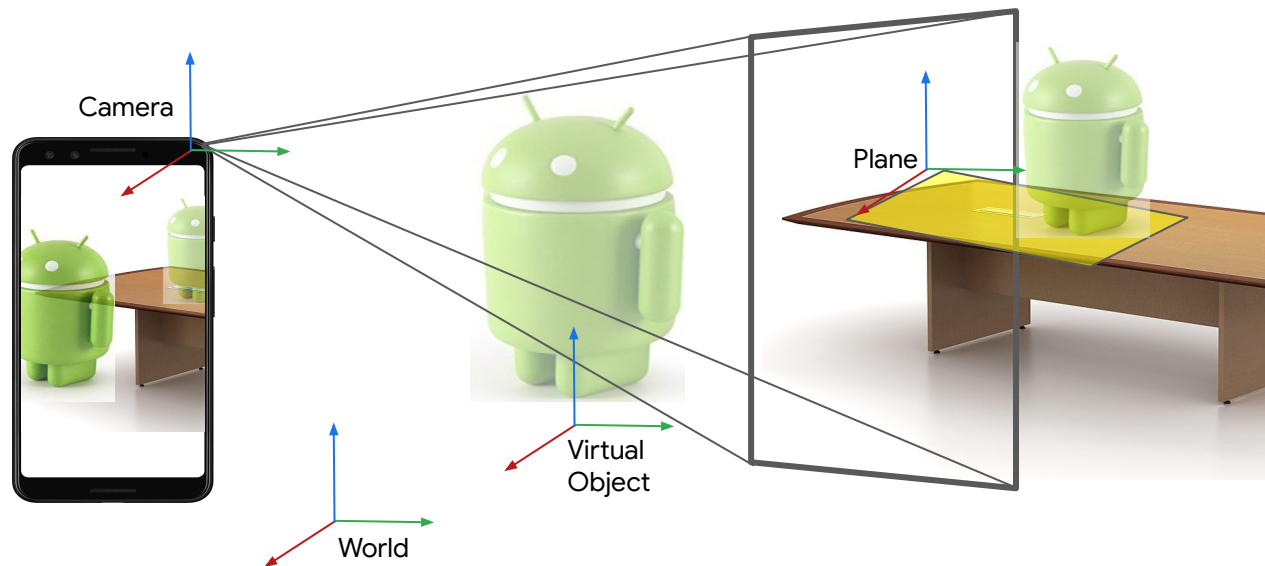
What We Can Do

Problems to solve

1. Where is the device?
2. Where are the real objects?
3. How will the virtual object look like?

Applications

1. Depth estimation
2. Motion tracking
3. Plane detection
4. Light estimation
5. Visual Simultaneous Localization and Mapping (vSLAM)



Closing Thoughts

- Pinhole Camera + Homogeneous Coordinates
- Camera Intrinsics + Extrinsics + Distortions
- Android APIs & Applications
- We want to hear your feedback!
- Join our developer forum for feedback and questions
 - <https://groups.google.com/a/android.com/g/camerax-developers>

Further Study

- <https://www.mathworks.com/help/vision/ug/camera-calibration.html>
- <https://developers.google.com/ar>
- <https://developer.apple.com/documentation/arkit/>
- [QR Decomposition of Projection Matrix](#)
- [Essential Matrix and Epipolar Constraint](#)
- [Quaternion to Rotation Matrix](#)

Thank you